

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL AND DEBUGGING

Oleh:

Muhammad Bukhari Fitri NIM. 2310817210015

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2024**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Bukhari Fitri
NIM : 2310817210015

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Ir. Eka Setya Wijaya, S.T., M.Kom
NIP. 198205082008011010

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code.....	6
B. Output Program	31
C. Pembahasan	39
D. Tautan Git	60
SOAL 2.....	61
A. Pembahasan	61

DAFTAR GAMBAR

Gambar 1. Contoh UI List	Error! Bookmark not defined.
Gambar 2. Contoh UI Detail	Error! Bookmark not defined.
Gambar 3. Tampilan Awal UI XML	31
Gambar 4. Tampilan Awal UI Landscape XML	31
Gambar 5. Tampilan Tombol Beli Buku Ditekan XML	32
Gambar 6. Tampilan Ketika Discroll XML	33
Gambar 7. Tampilan Tombol Detail Ditekan XML	34
Gambar 8. Tampilan Tombol Detail Ditekan Landscape XML	34
Gambar 9. Tampilan Awal UI Compose	35
Gambar 10. Tampilan Ketika Discroll Compose	36
Gambar 11. Tampilan Awal UI Landscape Compose	36
Gambar 12. Tampilan Tombol Beli Buku Ditekan Compose	37
Gambar 13. Tampilan Detail Compose	38
Gambar 14. Tampilan Detail Landscape Compose	38

DAFTAR TABEL

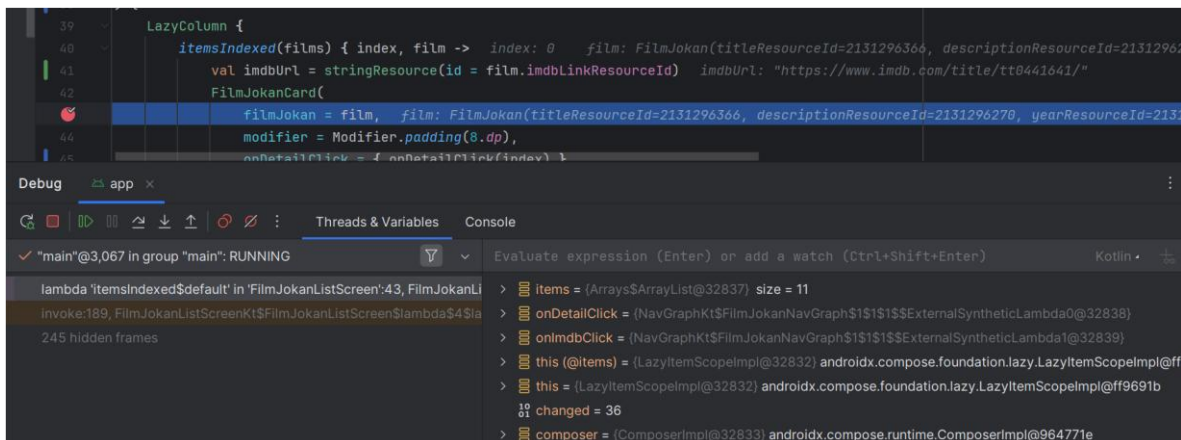
Tabel 1. Source Code MainActivity.kt XML	6
Tabel 2. Source Code Book.kt XML	7
Tabel 3. Source Code BookFragment.kt XML.....	7
Tabel 4. Source Code DetailFragment.kt XML	9
Tabel 5. Source Code MyItemRecyclerViewAdapter.kt XML	11
Tabel 6. Source Code activity_main.xml XML	13
Tabel 7. Source Code Jawaban Soal 1 activity_main XML. Error! Bookmark not defined.	
Tabel 8. Source Code fragment_book.xml XML	14
Tabel 9. Source Code fragment_detail.xml XML	14
Tabel 10. Source Code item_book.xml XML	17
Tabel 11. Source Code MainActivity.kt Compose.....	20
Tabel 12. Source Code Datasource.kt Compose	22
Tabel 13. Source Code Book.kt Compose.....	23
Tabel 14. Source Code BookDetailScreen.kt Compose	23
Tabel 15. Source Code BookListScreen.kt Compose.....	25

SOAL 1

Soal Praktikum:

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- Install dan gunakan library Timber untuk logging event berikut:
 - Log saat data item masuk ke dalam list
 - Log saat tombol Detail dan tombol Explicit Intent ditekan
 - Log data dari list yang dipilih ketika berpindah ke halaman Detail Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi.
- Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara



Gambar 1. Contoh Penggunaan Debugger

A. Source Code

XML:

MainActivity.kt

Tabel 1. Source Code MainActivity.kt XML

1	package com.example.booklist
2	
3	import android.os.Bundle
4	import androidx.activity.enableEdgeToEdge

5	import androidx.appcompat.app.AppCompatActivity
6	import androidx.core.view.ViewCompat
7	import androidx.core.view.WindowInsetsCompat
8	import com.example.booklist.databinding.ActivityMainBinding
9	
10	class MainActivity : AppCompatActivity() {
11	override fun onCreate(savedInstanceState: Bundle?) {
12	super.onCreate(savedInstanceState)
13	val binding =
14	ActivityMainBinding.inflate(layoutInflater)
15	setContentView(binding.root)
16	}

Book.kt

Tabel 2. Source Code Book.kt XML

1	package com.example.booklist
2	
3	import androidx.annotation.DrawableRes
4	import androidx.annotation.StringRes
5	
6	data class Book(
7	@StringRes val titleResourceId: Int,
8	@StringRes val yearResourceId: Int,
9	@StringRes val aboutResourceId: Int,
10	@StringRes val webUrlResourceId: Int,
11	@DrawableRes val imageResourceId: Int
12	)

BookFragment.kt

Tabel 3. Source Code BookFragment.kt XML

1	package com.example.booklist
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.fragment.app.viewModels
9	import androidx.lifecycle.LifecycleScope
10	import androidx.navigation.fragment.findNavController
11	import androidx.recyclerview.widget.LinearLayoutManager
12	import com.example.booklist.databinding.FragmentBookBinding
13	import kotlinx.coroutines.flow.collectLatest
14	import kotlinx.coroutines.launch
15	import timber.log.Timber
16	

```

17
18 class BookFragment : Fragment() {
19     private val viewModel: BookViewModel by viewModels {
20         BookViewModelFactory("MyCategory")
21     }
22
23     private var _binding: FragmentBookBinding? = null
24     private val binding get() = _binding!!
25
26     override fun onCreateView(inflater: LayoutInflater,
27 container: ViewGroup?, savedInstanceState: Bundle?): View {
28         _binding = FragmentBookBinding.inflate(inflater,
29 container, false)
30         return binding.root
31     }
32
33     override fun onViewCreated(view: View,
34 savedInstanceState: Bundle?) {
35         val recyclerView = binding.recyclerView
36         recyclerView.layoutManager =
37         LinearLayoutManager(requireContext())
38
39         lifecycleScope.launch {
40             viewModel.books.collectLatest { bookList ->
41                 recyclerView.adapter =
42                 BookAdapter(requireContext(), bookList) { selectedBook ->
43                     Timber.d("Tombol 'Tentang' ditekan untuk
44                     buku dengan judul ID: ${selectedBook.titleResourceId}")
45                     val bundle = Bundle().apply {
46                         putInt("titleRes",
47                             selectedBook.titleResourceId)
48                         putInt("yearRes",
49                             selectedBook.yearResourceId)
50                         putInt("aboutRes",
51                             selectedBook.aboutResourceId)
52                         putInt("imageResId",
53                             selectedBook.imageResourceId)
54                     }
55
56                     findNavController().navigate(R.id.action_bookFragment_to_detailFragment, bundle)
57                 }
58             }
59         }
60
61         override fun onDestroyView() {
62             super.onDestroyView()
63             _binding = null
64         }
65     }
66 }

```


DetailFragment.kt

Tabel 4. Source Code DetailFragment.kt XML

```
1 package com.example.booklist
2
3 import android.os.Bundle
4 import androidx.fragment.app.Fragment
5 import android.view.LayoutInflater
6 import android.view.View
7 import android.view.ViewGroup
8 import com.example.booklist.databinding.FragmentDetailBinding
9
10 class DetailFragment : Fragment() {
11
12     private var _binding: FragmentDetailBinding? = null
13     private val binding get() = _binding!!
14
15     override fun onCreateView(
16         inflater: LayoutInflater,
17         container: ViewGroup?,
18         savedInstanceState: Bundle?
19     ): View? {
20         _binding = FragmentDetailBinding.inflate(inflater,
21 container, false)
22         return binding.root
23     }
24
25     override fun onViewCreated(view: View, savedInstanceState:
26 Bundle?) {
27         val args = arguments
28         val titleRes = args?.getInt("titleRes") ?: 0
29         val yearRes = args?.getInt("yearRes") ?: 0
30         val aboutRes = args?.getInt("aboutRes") ?: 0
31         val imageResId = args?.getInt("imageResId") ?: 0
32
33         binding.bookTitle.setText(titleRes)
34         binding.bookYear.setText(yearRes)
35         binding.bookAbout.setText(aboutRes)
36         binding.bookImage.setImageResource(imageResId)
37     }
38
39     override fun onDestroyView() {
40         super.onDestroyView()
41         _binding = null
42     }
43 }
```

BookViewModel.kt

Tabel 5. Source Code BookViewModel.kt XML

1	<code>package com.example.booklist</code>
2	
3	<code>import androidx.lifecycle.ViewModel</code>
4	<code>import androidx.lifecycle.ViewModelProvider</code>
5	<code>import kotlinx.coroutines.flow.MutableStateFlow</code>
6	<code>import kotlinx.coroutines.flow.StateFlow</code>
7	<code>import timber.log.Timber</code>
8	
9	<code>class BookViewModel(category: String) : ViewModel() {</code>
10	
11	<code> private val _books =</code>
	<code>MutableStateFlow<List<Book>>(emptyList())</code>
12	<code> val books: StateFlow<List<Book>> = _books</code>
13	
14	<code> init {</code>
15	<code> loadBooks()</code>
16	<code> }</code>
17	
18	
19	<code> private fun loadBooks() {</code>
20	<code> _books.value = listOf(</code>
21	<code> Book(R.string.book_title1, R.string.book_year1,</code>
	<code>R.string.book_about1, R.string.book_webUrl1,</code>
	<code>R.drawable.image1),</code>
22	<code> Book(R.string.book_title2, R.string.book_year2,</code>
	<code>R.string.book_about2, R.string.book_webUrl2,</code>
	<code>R.drawable.image2),</code>
23	<code> Book(R.string.book_title3, R.string.book_year3,</code>
	<code>R.string.book_about3, R.string.book_webUrl3,</code>
	<code>R.drawable.image3),</code>
24	<code> Book(R.string.book_title4, R.string.book_year4,</code>
	<code>R.string.book_about4, R.string.book_webUrl4,</code>
	<code>R.drawable.image4),</code>
25	<code> Book(R.string.book_title5, R.string.book_year5,</code>
	<code>R.string.book_about5, R.string.book_webUrl5,</code>
	<code>R.drawable.image5),</code>
26	<code> Book(R.string.book_title6, R.string.book_year6,</code>
	<code>R.string.book_about6, R.string.book_webUrl6,</code>
	<code>R.drawable.image6),</code>
27	<code> Book(R.string.book_title7, R.string.book_year7,</code>
	<code>R.string.book_about7, R.string.book_webUrl7,</code>
	<code>R.drawable.image7),</code>
28	<code> Book(R.string.book_title8, R.string.book_year8,</code>
	<code>R.string.book_about8, R.string.book_webUrl8,</code>
	<code>R.drawable.image8)</code>
29	<code>)</code>
30	<code> Timber.i("Data buku berhasil dimuat:</code>
	<code>\${_books.value.size}")</code>
31	<code> }</code>
32	<code>}</code>

```

33
34 class BookViewModelFactory(private val category: String) :
35 ViewModelProvider.Factory {
36     override fun <T : ViewModel> create(modelClass:
37     Class<T>): T {
38         if
39         (modelClass.isAssignableFrom(BookViewModel::class.java)) {
40             @Suppress("UNCHECKED_CAST")
41             return BookViewModel(category) as T
42         }
43         throw IllegalArgumentException("Unknown ViewModel
44         class")
45     }
46 }

```

MyItemRecyclerViewAdapter.kt

Tabel 6. Source Code MyItemRecyclerViewAdapter.kt XML

```

1 package com.example.booklist
2
3 import android.content.Context
4 import android.content.Intent
5 import android.net.Uri
6 import android.view.LayoutInflater
7 import android.view.ViewGroup
8 import androidx.recyclerview.widget.RecyclerView
9 import com.example.booklist.databinding.ItemBookBinding
10 import timber.log.Timber
11
12 class BookAdapter(
13     private val context: Context,
14     private val books: List<Book>,
15     private val onAboutClick: (Book) -> Unit
16 ) : RecyclerView.Adapter<BookAdapter.BookViewHolder>() {
17
18     inner class BookViewHolder(val binding: ItemBookBinding)
19     : RecyclerView.ViewHolder(binding.root)
20
21     override fun onCreateViewHolder(parent: ViewGroup,
22     viewType: Int): BookViewHolder {
23         val binding =
24         ItemBookBinding.inflate(LayoutInflater.from(context), parent,
25         false)
26         return BookViewHolder(binding)
27     }
28
29     override fun onBindViewHolder(holder: BookViewHolder,
30     position: Int) {
31         val book = books[position]

```

27	<code>with(holder.binding) {</code>
28	<code>bookTitle.text =</code>
	<code>context.getString(book.titleResourceId)</code>
29	<code>bookYear.text =</code>
	<code>context.getString(book.yearResourceId)</code>
30	<code>bookAbout.text =</code>
	<code>context.getString(book.aboutResourceId)</code>
31	<code>bookImage.setImageResource(book.imageResourceId)</code>
32	
33	<code>btnBuy.setOnClickListener {</code>
34	<code>Timber.d("Tombol 'Beli' ditekan untuk URL ID:</code>
	<code>\${book.webUrlResourceId}")</code>
35	<code>val intent = Intent(Intent.ACTION_VIEW,</code>
	<code>Uri.parse(context.getString(book.webUrlResourceId)))</code>
36	<code>context.startActivity(intent)</code>
37	<code>}</code>
38	
39	<code>btnAbout.setOnClickListener {</code>
40	<code>Timber.d("Tombol 'Tentang' ditekan untuk</code>
	<code>buku: \${context.getString(book.titleResourceId)})</code>
41	<code>onAboutClick(book)</code>
42	<code>}</code>
43	<code>}</code>
44	<code>}</code>
45	
46	<code>override fun getItemCount() = books.size</code>
47	<code>}</code>

MyApp.kt

Tabel 7. Source Code MyApp.kt XML

1	<code>package com.example.booklist</code>
2	
3	<code>import android.app.Application</code>
4	<code>import timber.log.Timber</code>
5	
6	<code>class MyApp : Application() {</code>
7	<code>override fun onCreate() {</code>
8	<code>super.onCreate()</code>
9	<code>Timber.plant(Timber.DebugTree())</code>
10	<code>}</code>
11	<code>}</code>

activity_main.xml

Tabel 8. Source Code activity_main.xml XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/main"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".MainActivity">
9	
10	<FrameLayout
11	android:id="@+id/frame_layout"
12	android:layout_width="match_parent"
13	android:layout_height="match_parent"
14	app:layout_constraintBottom_toBottomOf="parent"
15	app:layout_constraintEnd_toEndOf="parent"
16	app:layout_constraintHorizontal_bias="1.0"
17	app:layout_constraintStart_toStartOf="parent"
18	app:layout_constraintTop_toTopOf="parent"
19	app:layout_constraintVertical_bias="0.0">
20	
21	<androidx.fragment.app.FragmentContainerView
22	android:id="@+id/nav_host_fragment"
23	android:name="androidx.navigation.fragment.NavHostFragment"
24	android:layout_width="match_parent"
25	android:layout_height="match_parent"
26	app:navGraph="@navigation/nav_graph"
27	app:defaultNavHost="true" />
28	
29	</FrameLayout>
30	</androidx.constraintlayout.widget.ConstraintLayout>

fragment_book.xml

Tabel 9. Source Code fragment_book.xml XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".BookFragment">
9	
10	<androidx.recyclerview.widget.RecyclerView
11	android:id="@+id/recyclerView"
12	android:layout_width="0dp"
13	android:layout_height="0dp"
14	app:layout_constraintTop_toTopOf="parent"
15	app:layout_constraintBottom_toBottomOf="parent"
16	app:layout_constraintStart_toStartOf="parent"
17	app:layout_constraintEnd_toEndOf="parent" />
18	
19	</androidx.constraintlayout.widget.ConstraintLayout>

fragment_detail.xml

Tabel 10. Source Code fragment_detail.xml XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:background="#1C1C1C"
8	android:padding="16dp">
9	<LinearLayout
10	android:layout_width="match_parent"
11	android:layout_height="wrap_content"
12	android:orientation="vertical">
13	
14	<androidx.constraintlayout.widget.ConstraintLayout
15	android:layout_width="match_parent"
16	android:layout_height="wrap_content">
17	
18	<androidx.cardview.widget.CardView
19	android:id="@+id/card_image"
20	android:layout_width="wrap_content"
21	android:layout_height="wrap_content"
22	app:cardCornerRadius="8dp"
23	app:layout_constraintTop_toTopOf="parent"
24	app:layout_constraintStart_toStartOf="parent"
25	app:layout_constraintEnd_toEndOf="parent">

```

26
27         <ImageView
28             android:id="@+id/bookImage"
29             android:layout_width="350dp"
30             android:layout_height="520dp"
31             android:scaleType="centerCrop"
32             android:src="@drawable/imagel" />
33     </androidx.cardview.widget.CardView>
34
35     <TextView
36         android:id="@+id/bookTitle"
37         android:layout_width="0dp"
38         android:layout_height="wrap_content"
39         android:layout_marginTop="23dp"
40         android:text="@string/book_title1"
41         android:textColor="@color/white"
42         android:textSize="24sp"
43         android:textStyle="bold"
44         app:layout_constraintTop_toBottomOf="@id/card_image"
45         app:layout_constraintStart_toStartOf="parent"
46         app:layout_constraintEnd_toStartOf="@id/bookYear"
47         app:layout_constraintHorizontal_weight="1"
48     />
49
50     <TextView
51         android:id="@+id/bookYear"
52         android:layout_width="wrap_content"
53         android:layout_height="wrap_content"
54         android:layout_marginTop="24dp"
55         android:text="@string/book_year1"
56         android:textColor="@color/white"
57         android:textSize="24sp"
58         app:layout_constraintTop_toBottomOf="@id/card_image"
59         app:layout_constraintEnd_toEndOf="parent"
60         app:layout_constraintStart_toEndOf="@id/bookTitle" />
61
62     <TextView
63         android:id="@+id/textTentang"
64         android:layout_width="wrap_content"
65         android:layout_height="wrap_content"
66         android:layout_marginTop="8dp"
67         android:text="@string/about"
68         android:textColor="@color/white"
69         android:textSize="14sp"
70         app:layout_constraintTop_toBottomOf="@id/bookTitle"
71         app:layout_constraintStart_toStartOf="parent" />
72
73     <TextView
74         android:id="@+id/bookAbout"
75         android:layout_width="0dp"
76         android:layout_height="wrap_content"
77         android:layout_marginTop="8dp"

```

77	<code>android:layout_marginBottom="8dp"</code>
78	<code>android:text="@string/book_about1"</code>
79	<code>android:textColor="@color/white"</code>
80	<code>android:textSize="14sp"</code>
81	<code>app:layout_constraintTop_toBottomOf="@id/textTentang"</code>
82	<code>app:layout_constraintStart_toStartOf="parent"</code>
83	<code>app:layout_constraintEnd_toEndOf="parent" /></code>
84	<code></androidx.constraintlayout.widget.ConstraintLayout></code>
85	
86	<code></LinearLayout></code>
87	<code></ScrollView></code>

item_book.xml

Tabel 11. Source Code item_book.xml XML


```

50         android:textColor="@color/white"
51         android:textSize="24sp"
52         android:textStyle="bold"
53     app:layout_constraintEnd_toStartOf="@+id/bookYear"
54     app:layout_constraintStart_toEndOf="@+id/card_image"
55         app:layout_constraintTop_toTopOf="parent" />
56
57     <TextView
58         android:id="@+id/bookYear"
59         android:layout_width="wrap_content"
60         android:layout_height="wrap_content"
61         android:layout_marginTop="16dp"
62         android:layout_marginEnd="16dp"
63         android:text="@string/book_year1"
64         android:textColor="@color/white"
65         app:layout_constraintEnd_toEndOf="parent"
66         app:layout_constraintTop_toTopOf="parent" />
67
68     <TextView
69         android:id="@+id/textTentang"
70         android:layout_width="wrap_content"
71         android:layout_height="0dp"
72         android:layout_marginStart="8dp"
73         android:layout_marginTop="24dp"
74         android:text="@string/about"
75         android:textColor="@color/white"
76         android:textSize="14sp"
77     app:layout_constraintEnd_toStartOf="@+id/bookAbout"
78     app:layout_constraintStart_toEndOf="@+id/card_image"
79     app:layout_constraintTop_toBottomOf="@+id/bookTitle" />
80
81     <TextView
82         android:id="@+id/bookAbout"
83         android:layout_width="184dp"
84         android:layout_height="wrap_content"
85         android:layout_marginStart="6dp"
86         android:layout_marginTop="24dp"
87         android:layout_marginEnd="9dp"
88         android:text="@string/book_about1"
89         android:textColor="@color/white"
90         android:textSize="14sp"
91         app:layout_constraintEnd_toEndOf="parent"
92     app:layout_constraintStart_toEndOf="@+id/textTentang"
93     app:layout_constraintTop_toBottomOf="@+id/bookTitle" />
94

```

95	<Button
96	android:id="@+id/btnBuy"
97	android:layout_width="wrap_content"
98	android:layout_height="wrap_content"
99	android:layout_marginStart="55dp"
100	android:layout_marginTop="27dp"
101	android:layout_marginBottom="8dp"
102	android:text="@string/beri_button"
103	android:textSize="14sp"
104	app:layout_constraintBottom_toBottomOf="parent"
105	app:layout_constraintEnd_toStartOf="@+id/btnAbout"
106	app:layout_constraintStart_toEndOf="@+id/card_image"
107	app:layout_constraintTop_toBottomOf="@+id/bookAbout" />
108	
109	<Button
110	android:id="@+id/btnAbout"
111	android:layout_width="wrap_content"
112	android:layout_height="wrap_content"
113	android:layout_marginStart="8dp"
114	android:layout_marginTop="27dp"
115	android:layout_marginEnd="16dp"
116	android:layout_marginBottom="8dp"
117	android:text="@string/detail_button"
118	android:textSize="14sp"
119	app:layout_constraintBottom_toBottomOf="parent"
120	app:layout_constraintEnd_toEndOf="parent"
121	app:layout_constraintStart_toEndOf="@+id/btnBuy"
122	app:layout_constraintTop_toBottomOf="@+id/bookAbout" />
123	</androidx.constraintlayout.widget.ConstraintLayout>
124	</androidx.cardview.widget.CardView>
125	</androidx.constraintlayout.widget.ConstraintLayout>

Compose:

MainActivity.kt

Tabel 12. Source Code MainActivity.kt Compose

1	<code>package com.example.booklist</code>
2	
3	<code>import android.os.Bundle</code>
4	<code>import androidx.activity.ComponentActivity</code>
5	<code>import androidx.activity.compose.setContent</code>

```

6  import androidx.activity.enableEdgeToEdge
7  import androidx.compose.foundation.layout.WindowInsets
8  import androidx.compose.foundation.layout.asPaddingValues
9  import androidx.compose.foundation.layout.calculateEndPadding
10 import
    androidx.compose.foundation.layout.calculateStartPadding
11 import androidx.compose.foundation.layout.fillMaxSize
12 import androidx.compose.foundation.layout.padding
13 import androidx.compose.foundation.layout.safeDrawing
14 import androidx.compose.foundation.layout.statusBarsPadding
15 import androidx.compose.material3.Surface
16 import androidx.compose.material3.Text
17 import androidx.compose.runtime.Composable
18 import androidx.compose.runtime.collectAsState
19 import androidx.compose.ui.Modifier
20 import androidx.compose.ui.platform.LocalLayoutDirection
21 import androidx.compose.runtime.getValue
22 import androidx.compose.ui.unit.dp
23 import androidx.lifecycle.viewmodel.compose.viewModel
24 import androidx.navigation.compose.NavHost
25 import androidx.navigation.compose.composable
26 import androidx.navigation.compose.rememberNavController
27 import com.example.booklist.screens.BookDetailScreen
28 import com.example.booklist.screens.BookListScreen
29 import com.example.booklist.screens.BookListViewModel
30 import com.example.booklist.ui.theme.BookListTheme
31
32 class MainActivity : ComponentActivity() {
33     override fun onCreate(savedInstanceState: Bundle?) {
34         super.onCreate(savedInstanceState)
35         enableEdgeToEdge()
36         setContent {
37             BookListTheme {
38                 BooksApp()
39             }
40         }
41     }
42 }
43
44 @Composable
45 fun BooksApp() {
46     val layoutDirection = LocalLayoutDirection.current
47     val navController = rememberNavController()
48     val viewModel: BookListViewModel = viewModel()
49     val bookList by viewModel.booksList.collectAsState()
50     val selectedBook by
        viewModel.selectedBook.collectAsState()
51
52     Surface(
53         modifier = Modifier
54             .fillMaxSize()
55             .statusBarsPadding()

```

```

56         .padding(
57             start =
WindowInsets.safeDrawing.asPaddingValues().calculateStartPadd
ing(layoutDirection),
58             end =
WindowInsets.safeDrawing.asPaddingValues().calculateEndPaddin
g(layoutDirection),
59         )
60     ) {
61         NavHost(navController = navController,
startDestination = "list") {
62             composable("list") {
63                 BookListScreen(bookList = bookList,
onDetailClick = { index ->
64                     viewModel.selectBook(bookList[index])
65                     navController.navigate("detail")
66                 })
67             }
68             composable("detail") { backStackEntry ->
69                 if (selectedBook != null) {
70                     BookDetailScreen(book = selectedBook!!)
71                 } else {
72                     Text("Buku tidak ditemukan", modifier =
Modifier.padding(16.dp))
73                 }
74             }
75         }
76     }
77 }
78 }

```

Datasource.kt

Tabel 13. Source Code Datasource.kt Compose

```

1 package com.example.booklist.data
2
3 import com.example.booklist.R
4 import com.example.booklist.model.Book
5
6 class Datasource {
7     fun loadBooks(): List<Book> {
8         return listOf<Book>{
9             Book(R.string.book_title1, R.string.book_year1,
R.string.book_about1, R.string.book_webUrl1,
R.drawable.image1) ,
10             Book(R.string.book_title2, R.string.book_year2,
R.string.book_about2, R.string.book_webUrl2,
R.drawable.image2),
11         }

```

12	Book(R.string.book_title3, R.string.book_year3, R.string.book_about3, R.string.book_webUrl3, R.drawable.image3),
13	Book(R.string.book_title4, R.string.book_year4, R.string.book_about4, R.string.book_webUrl4, R.drawable.image4),
14	Book(R.string.book_title5, R.string.book_year5, R.string.book_about5, R.string.book_webUrl5, R.drawable.image5),
15	Book(R.string.book_title6, R.string.book_year6, R.string.book_about6, R.string.book_webUrl6, R.drawable.image6),
16	Book(R.string.book_title7, R.string.book_year7, R.string.book_about7, R.string.book_webUrl7, R.drawable.image7),
17	Book(R.string.book_title8, R.string.book_year8, R.string.book_about8, R.string.book_webUrl8, R.drawable.image8)
18	)
19	}
	}

Book.kt

Tabel 14. Source Code Book.kt Compose

1	package com.example.booklist.model
2	
3	import androidx.annotation.DrawableRes
4	import androidx.annotation.StringRes
5	
6	data class Book(7 @StringRes val titleResourceId: Int, 8 @StringRes val yearResourceId: Int, 9 @StringRes val aboutResourceId: Int, 10 @StringRes val webUrlResourceId: Int, 11 @DrawableRes val imageResourceId: Int 12)

BookDetailScreen.kt

Tabel 15. Source Code BookDetailScreen.kt Compose

1	package com.example.booklist.screens
2	
3	import android.content.res.Configuration
4	import androidx.compose.foundation.Image
5	import androidx.compose.foundation.layout.Arrangement
6	import androidx.compose.foundation.layout.Column
7	import androidx.compose.foundation.layout.Row
8	import androidx.compose.foundation.layout.Spacer

```

9      import androidx.compose.foundation.layout.fillMaxSize
10     import androidx.compose.foundation.layout.fillMaxWidth
11     import androidx.compose.foundation.layout.height
12     import androidx.compose.foundation.layout.padding
13     import androidx.compose.foundation.layout.size
14     import androidx.compose.foundation.layout.width
15     import androidx.compose.foundation.rememberScrollState
16     import androidx.compose.foundation.shape.RoundedCornerShape
17     import androidx.compose.foundation.verticalScroll
18     import androidx.compose.material3.MaterialTheme
19     import androidx.compose.material3.Text
20     import androidx.compose.runtime.Composable
21     import androidx.compose.ui.Alignment
22     import androidx.compose.ui.Modifier
23     import androidx.compose.ui.draw.clip
24     import androidx.compose.ui.layout.ContentScale
25     import androidx.compose.ui.platform.LocalConfiguration
26     import androidx.compose.ui.platform.LocalContext
27     import androidx.compose.ui.res.painterResource
28     import androidx.compose.ui.text.style.TextOverflow
29     import androidx.compose.ui.unit.dp
30     import androidx.compose.ui.unit.sp
31     import com.example.booklist.model.Book
32     import timber.log.Timber
33
34     @Composable
35     fun BookDetailScreen(book: Book) {
36         Timber.i("Detail buku ditampilkan:
37         ${LocalContext.current.getString(book.titleResourceId)}")
38         Column(
39             modifier = Modifier
40                 .fillMaxSize()
41                 .padding(16.dp)
42                 .verticalScroll(rememberScrollState())
43         ) {
44             Image(
45                 painter =
46                 painterResource(book.imageResourceId),
47                 contentDescription = null,
48                 modifier = Modifier
49                     .size(width = 380.dp, height = 550.dp)
50                     .clip(RoundedCornerShape(16.dp))
51                     .align(Alignment.CenterHorizontally),
52                 contentScale = ContentScale.Crop
53             )
54
55             Spacer(modifier = Modifier.height(16.dp))
56
57             Column(
58                 modifier = Modifier.fillMaxWidth(),
59                 horizontalAlignment = Alignment.Start
60             ) {

```


59	Row(horizontalArrangement =
	Arrangement.SpaceBetween, modifier =
	Modifier.fillMaxWidth()) {
60	Text(
61	text =
	LocalContext.current.getString(book.titleResourceId),
62	style =
	MaterialTheme.typography.labelLarge,
63	fontSize = 26.sp
64	)
65	Text(
66	text =
	LocalContext.current.getString(book.yearResourceId),
67	style =
	MaterialTheme.typography.bodyMedium,
68	fontSize = 20.sp
69	)
70	}
71	Spacer(modifier = Modifier.height(8.dp))
72	Text("Tentang:", style =
	MaterialTheme.typography.bodySmall)
73	Text(
74	text =
	LocalContext.current.getString(book.aboutResourceId),
75	style =
	MaterialTheme.typography.bodySmall,
76	fontSize = 14.sp
77	)
78	}
79	}
80	}

BookListScreen.kt

Tabel 16. Source Code BookListScreen.kt Compose

1	<i>package com.example.booklist.screens</i>
2	
3	<i>import android.content.Intent</i>
4	<i>import android.net.Uri</i>
5	<i>import android.util.Log</i>
6	<i>import androidx.compose.foundation.Image</i>
7	<i>import androidx.compose.foundation.layout.Arrangement</i>
8	<i>import androidx.compose.foundation.layout.Column</i>
9	<i>import androidx.compose.foundation.layout.Row</i>
10	<i>import androidx.compose.foundation.layout.Spacer</i>
11	<i>import androidx.compose.foundation.layout.fillMaxWidth</i>
12	<i>import androidx.compose.foundation.layout.height</i>
13	<i>import androidx.compose.foundation.layout.padding</i>
14	<i>import androidx.compose.foundation.layout.size</i>
15	<i>import androidx.compose.foundation.layout.width</i>

```

16 import androidx.compose.foundation.lazy.LazyColumn
17 import androidx.compose.foundation.lazy.items
18 import androidx.compose.foundation.lazy.itemsIndexed
19 import androidx.compose.foundation.shape.RoundedCornerShape
20 import androidx.compose.material3.Button
21 import androidx.compose.material3.Card
22 import androidx.compose.material3.MaterialTheme
23 import androidx.compose.material3.Text
24 import androidx.compose.runtime.Composable
25 import androidx.compose.ui.Alignment
26 import androidx.compose.ui.Modifier
27 import androidx.compose.ui.draw.clip
28 import androidx.compose.ui.layout.ContentScale
29 import androidx.compose.ui.platform.LocalContext
30 import androidx.compose.ui.res.painterResource
31 import androidx.compose.ui.res.stringResource
32 import androidx.compose.ui.text.style.TextOverflow
33 import androidx.compose.ui.tooling.preview.Preview
34 import androidx.compose.ui.unit.dp
35 import androidx.compose.ui.unit.sp
36 import com.example.booklist.BooksApp
37 import com.example.booklist.model.Book
38 import timber.log.Timber
39
40 @Composable
41 fun BookListScreen(bookList: List<Book>, onDetailClick:
  (Int) -> Unit, modifier: Modifier = Modifier) {
42     LazyColumn(modifier = modifier) {
43         itemsIndexed(bookList) { index, book ->
44             Timber.d("Tombol 'Detail' ditekan untuk
menampilkan: ${book.titleResourceId}")
45             BookCard(book = book, onDetailClick = {
46                 onDetailClick(index)
47             })
48         }
49     }
50 }
51
52 @Composable
53 fun BookCard(book: Book, onDetailClick: () -> Unit,
modifier: Modifier = Modifier) {
54     Card(
55         modifier = modifier
56             .fillMaxWidth()
57             .padding(8.dp),
58         shape = RoundedCornerShape(16.dp)
59     ) {
60         val context = LocalContext.current
61         Row(
62             modifier = Modifier
63                 .padding(16.dp)
64                 .fillMaxWidth()

```

```

65         ) {
66             Image(
67                 painter =
68                 painterResource(book.imageResourceId),
69                 contentDescription =
70                 stringResource(book.aboutResourceId),
71                 modifier = Modifier
72                     .size(width = 120.dp, height = 200.dp)
73                     .clip(RoundedCornerShape(12.dp))
74                     .align(Alignment.CenterVertically),
75                 contentScale = ContentScale.Crop
76             )
77             Spacer(modifier = Modifier.width(12.dp))
78             Column(
79                 modifier = Modifier
80                     .weight(1f)
81                     .align(Alignment.CenterVertically)
82             ) {
83                 Row(
84                     modifier = Modifier.fillMaxWidth(),
85                     horizontalArrangement =
86                     Arrangement.SpaceBetween
87                 ) {
88                     Text(
89                         text =
90                         context.getString(book.titleResourceId),
91                         modifier = Modifier.weight(1f),
92                         style =
93                         MaterialTheme.typography.labelLarge,
94                         overflow = TextOverflow.Ellipsis,
95                         fontSize = 26.sp
96                     )
97                     Text(
98                         text =
99                         context.getString(book.yearResourceId),
100                         style =
101                         MaterialTheme.typography.bodyMedium,
102                         fontSize = 14.sp
103                     )
104                 }
105                 Spacer(modifier = Modifier.height(20.dp))
106                 Row {
107                     Text(
108                         text = "Tentang: ",
109                         style =
110                         MaterialTheme.typography.bodySmall,
111                         fontSize = 14.sp
112                     )
113                     Text(
114                         text =
115                         context.getString(book.aboutResourceId),

```

```

108         style =
109             MaterialTheme.typography.bodySmall,
110             fontSize = 14.sp
111     )
112     }
113     Spacer(modifier = Modifier.height(16.dp))
114     Row(modifier = Modifier.fillMaxWidth(),
115         horizontalArrangement =
116         Arrangement.End) {
117         Button(onClick = {
118             val webUrl =
119             context.getString(book.webUrlResourceId)
120             Timber.tag("BookCard").d("URL yang
121             akan dibuka: $webUrl")
122             val intent =
123             Intent(Intent.ACTION_VIEW, Uri.parse(webUrl))
124             context.startActivity(intent)
125         }) {
126             Text("Beli Buku")
127         }
128         Spacer(modifier = Modifier.width(8.dp))
129         Button(onClick = {
130             onDetailClick()
131         }) {
132             Text("Detail")
133         }
134     }
135 }

```

BookListViewModel.kt

Tabel 17. Source Code BookListViewModel.kt Compose

```

1 package com.example.booklist.screens
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.viewModelScope
5 import com.example.booklist.data.Datasource
6 import com.example.booklist.model.Book
7 import kotlinx.coroutines.flow.MutableStateFlow
8 import kotlinx.coroutines.flow.StateFlow
9 import kotlinx.coroutines.launch
10 import timber.log.Timber
11
12 class BookListViewModel(private val source: String) :
13     ViewModel() {

```

13	private val _booksList =
	MutableStateFlow<List<Book>>(emptyList())
14	val booksList: StateFlow<List<Book>> = _booksList
15	
16	private val _selectedBook =
	MutableStateFlow<Book?>(null)
17	val selectedBook: StateFlow<Book?> = _selectedBook
18	
19	init {
20	Timber.d("Sumber data ViewModel: \$source")
21	loadBooks()
22	}
23	
24	private fun loadBooks() {
25	viewModelScope.launch {
26	val books = Datasource().loadBooks()
27	_booksList.value = books
28	Timber.d("Data buku berhasil dimuat:
	\${books.size} item")
29	}
30	}
31	
32	fun selectBook(book: Book) {
33	_selectedBook.value = book
34	Timber.d("Buku yang dipilih:
	\${book.titleResourceId}")
35	}
36	}

BookListViewModelFactory.kt

Tabel 18. Source Code BookListViewModelFactory.kt Compose

1	package com.example.booklist.screens
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class BookListViewModelFactory(private val source: String)
	: ViewModelProvider.Factory {
7	override fun <T : ViewModel> create(modelClass:
	Class<T>): T {
8	if
	(modelClass.isAssignableFrom((BookListViewModel::class.java
	))) {
9	@Suppress("UNCHECKED_CAST")
10	return BookListViewModel(source) as T
11	}
12	throw IllegalArgumentException("Unknown ViewModel
	class")

13	}
14	}

MyApplication.kt

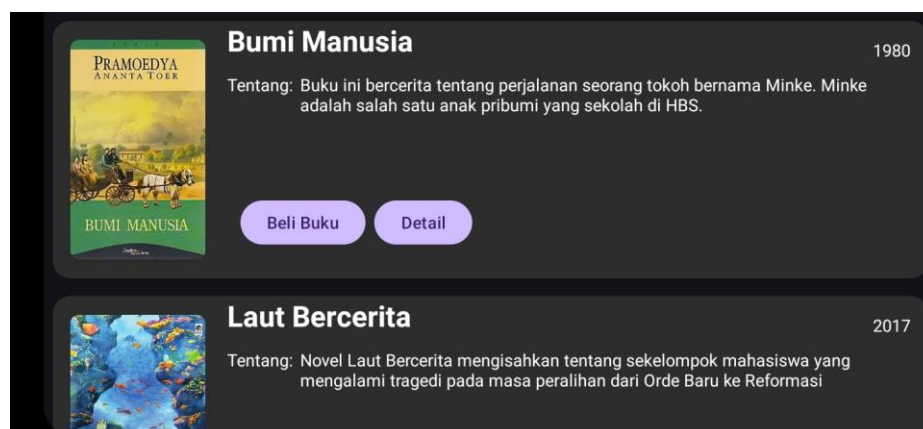
Tabel 19. Source CodeMyApplication.kt Compose

1	package com.example.booklist
2	
3	import android.app.Application
4	import timber.log.Timber
5	
6	
7	class MyApplication : Application() {
8	override fun onCreate() {
9	super.onCreate()
10	
11	if (BuildConfig.DEBUG) {
12	Timber.plant (Timber.DebugTree())
13	}
14	}
15	}

B. Output Program



Gambar 2. Tampilan Awal UI XML



Gambar 3. Tampilan Awal UI Landscape XML



Gambar 4. Tampilan Tombol Beli Buku Ditekan XML



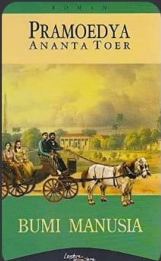
Gambar 5. Tampilan Ketika Discroll XML



Gambar 6. Tampilan Tombol Detail Ditekan XML



Gambar 7. Tampilan Tombol Detail Ditekan Landscape XML



Bumi Manusia

1980

Tentang: Buku ini bercerita tentang perjalanan seorang tokoh bernama Minke. Minke adalah salah satu anak pribumi yang sekolah di HBS.

[Beli Buku](#)
[Detail](#)



Laut Bercerita

2017

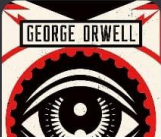
Tentang: Novel Laut Bercerita mengisahkan tentang sekelompok mahasiswa yang mengalami tragedi pada masa peralihan dari Orde Baru ke Reformasi

[Beli Buku](#)
[Detail](#)

1984

1949

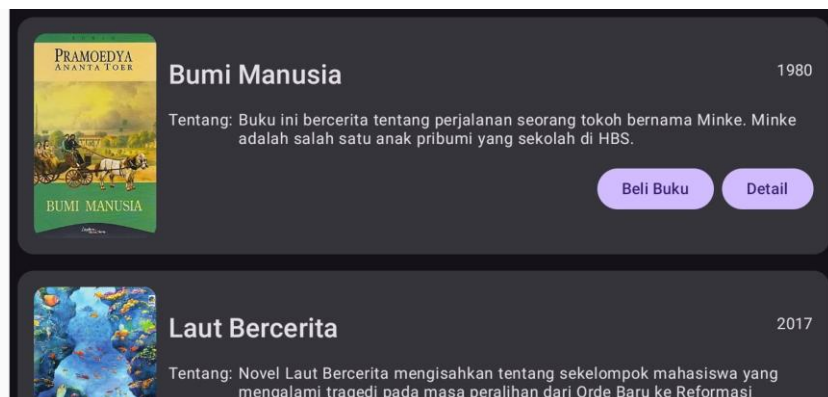
Tentang: Dalam dunia yang penuh pengawasan dan propaganda ini, Partai mengendalikan semua aspek kehidupan warganya. Kehidupan pribadi dan pemikiran



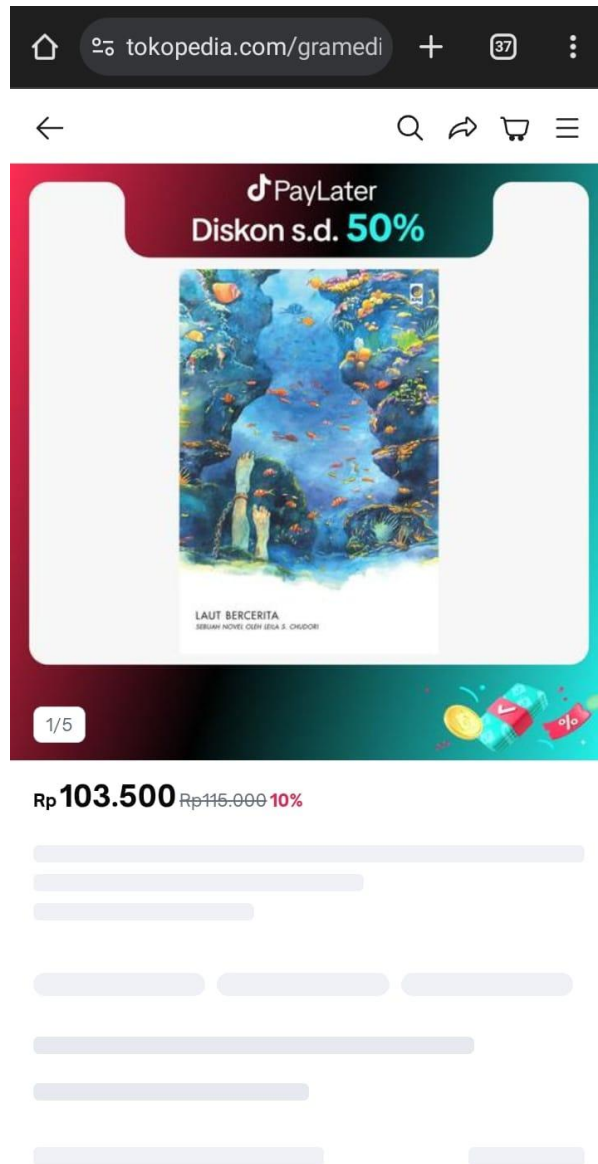
Gambar 8. Tampilan Awal UI Compose



Gambar 9. Tampilan Ketika Discroll Compose



Gambar 10. Tampilan Awal UI Landscape Compose



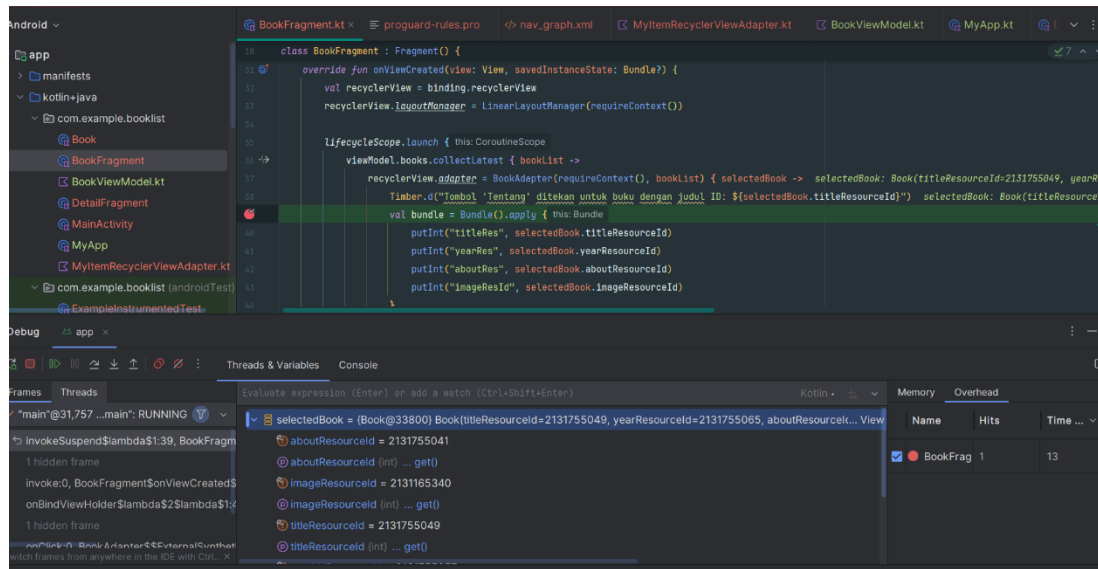
Gambar 11. Tampilan Tombol Beli Buku Ditekan Compose



Gambar 12. Tampilan Detail Compose



Gambar 13. Tampilan Detail Landscape Compose



Gambar 14. Penggunaan Debugger

C. Pembahasan

XML:

MainActivity.kt:

Pada baris [1], dideklarasikan nama package file Kotlin

Pada baris [3], diimport bundle untuk menyimpan data sementara antar lifecycle seperti onCreate, onSaveInstanceState, dll.

Pada baris [4], diimport enableEdgeToEdge agar konten bisa tampil sampai ke pinggir layar.

Pada baris [5], diimport AppCompatActivity superclass untuk activity yang menggunakan fitur-fitur dari AndroidX.

Pada baris [6], diimport ViewCompat digunakan untuk manipulasi property View.

Pada baris [7], diimport WindowInsetsCompat untuk mengatur tata letak yang mempertimbangkan status bar dan navigation bar.

Pada baris [8], diimport ActivityMainBinding class binding otomatis yang dihasilkan dari file XML activity_main.xml.

Pada baris [10], mendeklarasikan class MainActivity yang merupakan turunan dari AppCompatActivity sebagai titik awal saat aplikasi dijalankan.

Pada baris [11], mendefinisikan method onCreate, yaitu fungsi lifecycle yang dipanggil saat activity pertama kali dibuat.

Pada baris [12], memanggil implementasi onCreate di superclass untuk memastikan sistem Android tetap menjalankan proses inisialisasi standar activity.

Pada baris [13], membuat objek binding dengan cara menginflate layout XML yaitu activity_main.xml agar bisa diakses lewat kode Kotlin.

Pada baris [14], menetapkan root dari layout yang diinflate sebagai tampilan utama activity.

Book.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3], diimport DrawableRes untuk anotasi @DrawableRes yang menandakan bahwa parameter bertipe Int tersebut adalah resource drawable atau gambar.

Pada baris [4], diimport StringRes untuk anotasi @StringRes yang menandakan bahwa parameter Int tersebut adalah resource string dari folder res/values/strings.xml.

Pada baris [6-11], mendefinisikan class data Book untuk menyimpan data buku. @StringRes val titleResourceId: Int adalah properti untuk menyimpan ID dari resource judul buku. Anotasi @StringRes memberi tahu Android Studio bahwa nilai ini harus berasal dari resource string (R.string.nama_judul). @StringRes val yearResourceId: Int adalah ID resource string untuk tahun terbit buku (misal "2022"). @StringRes val aboutResourceId: Int menyimpan ID resource string yang menjelaskan isi buku atau deskripsinya. @StringRes val webViewResourceId: Int menyimpan ID resource string berupa URL website (untuk beli buku). @DrawableRes val imageResourceId: Int menyimpan ID resource gambar (drawable) untuk sampul buku. Anotasi @DrawableRes menjamin bahwa nilai ini berasal dari folder res/drawable.

BookFragment.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–15], diimport berbagai komponen penting yang dibutuhkan untuk menjalankan fungsi fragment ini. Bundle digunakan untuk mengirim data antar fragment. LayoutInflater, View, dan ViewGroup digunakan untuk mengatur tampilan layout. Fragment adalah class dasar untuk membuat fragment di Android. viewModels digunakan untuk menginisialisasi ViewModel di dalam fragment. lifecycleScope digunakan untuk menjalankan coroutine yang mengikuti lifecycle fragment. findNavController() adalah fungsi untuk melakukan navigasi

antar fragment dengan Jetpack Navigation. `LinearLayoutManager` digunakan untuk menampilkan data list secara vertikal di dalam `RecyclerView`. `FragmentBookBinding` merupakan class binding otomatis yang dihasilkan dari file XML `fragment_book.xml`, sehingga kita bisa mengakses komponen XML-nya langsung dari Kotlin. `collectLatest` digunakan untuk mengambil data terbaru dari `StateFlow`. `launch` digunakan untuk memulai coroutine. Timber adalah library logging yang digunakan untuk mencatat event dalam logcat. Pada baris [18], mendeklarasikan class fragment.

Pada baris [19-21], mendeklarasikan `ViewModel` dengan nama `viewModel` bertipe `BookViewModel`. `ViewModel` ini diinisialisasi menggunakan delegasi by `viewModels` dan dibantu oleh `BookViewModelFactory` dengan parameter kategori “MyCategory”.

Pada baris [23], menyimpan binding dengan tipe nullable untuk menghindari memory leak. Pada baris [24], memberi akses non null ke binding saat view aktif.

Pada baris [26–29], method `onCreateView()` digunakan untuk menginflate layout `fragment_book.xml` menjadi view, lalu `FragmentBookBinding.inflate()` menghubungkan dengan `ViewBinding`. Return `binding.root` untuk mengembalikan view utama fragment.

Pada baris [31–49], method `onViewCreated()` dipanggil setelah layout fragment berhasil ditampilkan di layar. Di dalamnya, pertama-tama diambil referensi ke `RecyclerView` menggunakan `binding.recyclerView`. Kemudian, `RecyclerView` dikonfigurasi untuk menampilkan daftar item secara vertikal dengan menggunakan `LinearLayoutManager`. Setelah itu, diluncurkan coroutine untuk mengamati `StateFlow` dari `ViewModel`. Adapter `BookAdapter` diset dengan data dari `bookList` yang berasal dari `ViewModel`. Adapter juga menerima lambda (callback) yang akan dipanggil saat pengguna mengklik tombol tentang pada salah satu buku. Ketika tombol ditekan, dicetak log menggunakan Timber.d yang berisi ID judul buku. Setelah itu, dibuat `Bundle` berisi informasi judul, tahun, deskripsi, dan gambar buku. `Bundle` ini dikirimkan ke `DetailFragment` menggunakan perintah navigasi `findNavController().navigate()`.

Pada baris [51–55], method `onDestroyView()` dipanggil saat fragment akan dihancurkan (misalnya ketika berpindah ke fragment lain). Di sini, `_binding` diset ke null untuk menghapus referensi ke view yang sudah tidak aktif, sehingga membantu mencegah terjadinya memory leak.

DetailFragment.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–8], diimport komponen-komponen penting yang dibutuhkan dalam fragment ini. Bundle digunakan untuk menerima data yang dikirim dari fragment sebelumnya. Fragment adalah superclass yang digunakan untuk membuat fragment. LayoutInflater, View, dan ViewGroup digunakan untuk mengelola dan menampilkan tampilan (layout) fragment. Terakhir, FragmentDetailBinding adalah class ViewBinding yang secara otomatis dihasilkan dari file XML fragment_detail.xml.

Pada baris [10], dideklarasikan class DetailFragment yang merupakan turunan dari Fragment. Class ini bertugas menampilkan detail buku yang dipilih oleh pengguna dari daftar buku.

Pada baris [12], dibuat properti _binding yang bertipe nullable dari FragmentDetailBinding. Properti ini akan menyimpan objek binding yang menghubungkan komponen layout dengan kode Kotlin, dan dibuat nullable agar bisa dihapus saat view dihancurkan untuk mencegah memory leak.

Pada baris [13], dibuat properti binding yang bersifat non-nullable dengan cara memaksa akses ke _binding menggunakan untuk mempermudah akses ke komponen layout selama fragment masih dalam keadaan aktif.

Pada baris [15–22], method onCreateView() digunakan untuk menginflate layout fragment_detail.xml menggunakan FragmentDetailBinding.inflate(). Hasil inflate disimpan ke _binding, lalu binding.root dikembalikan sebagai tampilan utama dari fragment ini.

Pada baris [24–35], method onViewCreated() dijalankan ketika layout fragment sudah selesai ditampilkan. Di sini, kita mengambil data yang dikirim dari BookFragment menggunakan objek arguments. Kemudian, masing-masing data diambil dari Bundle dengan kunci "titleRes", "yearRes", "aboutRes", dan "imageResId". Jika datanya tidak tersedia, maka akan digantikan dengan nilai default 0. Setelah itu, data tersebut ditampilkan ke layout. judul buku di-set ke bookTitle, tahun terbit ke bookYear, deskripsi ke bookAbout, dan gambar sampul ke bookImage.

Pada baris [37–41], method onDestroyView() dipanggil saat view fragment akan dihancurkan. Di sini, _binding diset ke null untuk membersihkan referensi terhadap view dan mencegah terjadinya memory leak.

BookViewModel.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–7], diimport berbagai komponen penting yang dibutuhkan untuk mendefinisikan ViewModel. ViewModel digunakan untuk menyimpan data yang terkait dengan UI dan bertahan saat konfigurasi berubah (seperti rotasi layar). ViewModelProvider digunakan untuk membuat ViewModel menggunakan factory. MutableStateFlow dan StateFlow digunakan untuk menyimpan dan mengamati data secara reaktif dalam arsitektur berbasis coroutine. Timber adalah library logging yang digunakan untuk mencatat aktivitas dalam aplikasi.

Pada baris [9], dideklarasikan class BookViewModel yang merupakan subclass dari ViewModel. ViewModel ini digunakan untuk menyimpan dan mengelola daftar buku dalam aplikasi. Class ini memiliki satu parameter bernama category bertipe String yang akan diterima melalui factory.

Pada baris [11], dibuat variabel privat _books bertipe MutableStateFlow dengan tipe data List<Book>. Variabel ini menyimpan daftar buku yang nantinya akan diobservasi oleh UI.

Pada baris [12], dibuat properti books yang merupakan StateFlow<List<Book>> dan merepresentasikan data yang bisa diamati dari luar class, namun hanya bisa diubah dari dalam ViewModel.

Pada baris [14-16], dibuat blok init yang akan dijalankan secara otomatis saat ViewModel dibuat. Di dalam blok ini, dipanggil fungsi loadBooks() untuk memuat data awal ke dalam daftar buku.

Pada baris [19–32], didefinisikan fungsi privat loadBooks() yang akan menetapkan isi dari daftar buku _books. Di dalamnya, dimasukkan delapan objek Book yang setiap parameternya mengambil data dari R.string dan R.drawable. Semua data diambil dari file resources seperti strings.xml dan drawable. Setelah data dimuat, dicetak log menggunakan Timber.i() yang menampilkan jumlah buku yang berhasil dimuat.

Pada baris [34], dideklarasikan class BookViewModelFactory yang merupakan subclass dari ViewModelProvider.Factory. Factory ini digunakan untuk membuat instance BookViewModel yang memerlukan parameter tambahan, yaitu category.

Pada baris [35], fungsi create() dioverride dari ViewModelProvider.Factory. Fungsi ini akan dipanggil secara otomatis saat kita memanggil by viewModels { } di dalam fragment.

Pada baris [36–39], dilakukan pengecekan apakah modelClass merupakan subclass dari BookViewModel. Jika iya, maka dibuat instance baru BookViewModel dengan parameter category, lalu dikembalikan sebagai tipe generic T dengan menggunakan @SuppressWarnings("UNCHECKED_CAST") untuk menghindari warning cast.

Pada baris [40], jika tipe ViewModel yang diminta tidak dikenali, maka akan dilemparkan exception IllegalArgumentException.

MyItemRecyclerViewAdapter.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–10], diimport berbagai komponen yang dibutuhkan dalam pembuatan adapter RecyclerView. Context dibutuhkan untuk akses ke resource aplikasi. Intent dan Uri digunakan untuk membuka link eksternal (seperti membuka halaman web). LayoutInflater dipakai untuk meng-inflate layout XML ke bentuk View. ViewGroup digunakan sebagai induk tampilan item. RecyclerView adalah komponen utama untuk membuat daftar yang bisa digulir. Terakhir, ItemBookBinding adalah class binding otomatis yang dihasilkan dari file item_book.xml. Timber digunakan sebagai alat untuk mencatat aktivitas ke logcat.

Pada baris [12–16], dideklarasikan class BookAdapter sebagai turunan dari RecyclerView.Adapter. Class ini bertugas menyediakan data dan tampilan untuk setiap item buku di dalam RecyclerView. Adapter ini memiliki tiga parameter: context untuk akses ke resource dan layout inflater, books yaitu list data buku yang akan ditampilkan, dan onAboutClick yaitu fungsi callback yang dijalankan ketika tombol "About" pada suatu item diklik.

Pada baris [18], dibuat inner class BookViewHolder yang bertugas menyimpan binding ke tampilan setiap item buku. Class ini merupakan turunan dari RecyclerView.ViewHolder, dan menggunakan binding.root sebagai tampilan utama untuk setiap elemen daftar.

Pada baris [20–23], diimplementasikan method onCreateViewHolder() yang bertugas membuat ViewHolder baru. Method ini meng-inflate layout item_book.xml menggunakan ItemBookBinding, lalu membungkusnya dalam objek BookViewHolder yang dikembalikan.

Pada baris [25–44], diimplementasikan method onBindViewHolder() yang bertugas menghubungkan data buku dengan tampilan. Data buku diambil berdasarkan posisi (index)

dalam list. Kemudian, nilai dari setiap properti buku ditampilkan pada komponen UI: judul, tahun, deskripsi, dan gambar buku. Untuk tombol "Buy", ketika diklik, akan dicetak log menggunakan Timber, lalu dibuat Intent dengan ACTION_VIEW dan membuka URL website buku di browser. Untuk tombol "About", ketika diklik, akan dicetak log berisi judul buku menggunakan Timber, lalu memanggil fungsi onAboutClick() dengan parameter objek Book yang sesuai.

Pada baris [46], diimplementasikan method getItemCount() yang mengembalikan jumlah total item dalam list buku. Method ini digunakan oleh RecyclerView untuk mengetahui berapa banyak data yang harus ditampilkan.

MyApp.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–4], diimport class-class penting yang dibutuhkan untuk mendefinisikan class aplikasi. Application adalah superclass dari seluruh komponen aplikasi Android dan digunakan untuk mengelola konfigurasi global aplikasi. Timber adalah library logging yang digunakan untuk mencatat log ke Logcat selama pengembangan dan debugging.

Pada baris [6], dideklarasikan class MyApp sebagai turunan dari class Application. Class ini digunakan untuk menginisialisasi konfigurasi global saat aplikasi pertama kali dijalankan.

Pada baris [7–10], dioverride method onCreate() yang akan dipanggil secara otomatis saat aplikasi dimulai. Di dalam method ini, dipanggil Timber.plant(Timber.DebugTree()) untuk menanam DebugTree yang memungkinkan kita menggunakan fungsi logging Timber seperti Timber.d() atau Timber.i() selama proses debugging aplikasi.

activity_main.xml:

Pada baris [1], terdapat versi XML dan encoding yang digunakan

Pada baris [2–30], dideklarasikan root layout yaitu ConstraintLayout, yang berasal dari library androidx.constraintlayout.widget.ConstraintLayout. Layout ini memungkinkan pengaturan posisi elemen berdasarkan batas (constraint) antar komponen atau terhadap parent. Di dalamnya juga terdapat deklarasi xmlns (XML namespace) untuk android, app, dan tools, yang masing-masing digunakan untuk atribut standar Android, atribut khusus dari

AndroidX, dan preview di editor. `tools:context=".MainActivity"` digunakan hanya di Android Studio untuk menunjukkan bahwa layout ini digunakan oleh MainActivity.

Pada baris [10–29], didefinisikan `FrameLayout` dengan ID `@+id/frame_layout` sebagai wadah utama yang mengisi seluruh layar (`match_parent` untuk lebar dan tinggi). Komponen ini diberi constraint agar menempel ke semua sisi parent `ConstraintLayout`, yaitu Top, Bottom, Start, dan End. Ini berarti layout akan memenuhi seluruh layar aplikasi. Bias horizontal dan vertikal juga diset (meskipun tidak wajib) untuk mengatur keseimbangan layout di tengah.

Pada baris [21–27], di dalam `FrameLayout`, terdapat komponen `FragmentContainerView` yang merupakan tempat untuk menampilkan fragment. Komponen ini diberi ID `@+id/nav_host_fragment`, dan diberi atribut `android:name="androidx.navigation.fragment.NavHostFragment"` yang menunjukkan bahwa fragment ini digunakan untuk sistem navigasi. Atribut `app:navGraph="@navigation/nav_graph"` menghubungkan fragment ini dengan file navigasi `nav_graph.xml`, yang berisi pengaturan alur navigasi antar fragment. Atribut `app:defaultNavHost="true"` memastikan bahwa `NavHostFragment` ini akan menangani navigasi kembali (Back) secara otomatis dari sistem.

fragment_book.xml:

Pada baris [1], terdapat versi XML dan encoding yang digunakan

Pada baris [2–19], dideklarasikan root layout menggunakan `ConstraintLayout` dari AndroidX (`androidx.constraintlayout.widget.ConstraintLayout`). Layout ini memungkinkan pengaturan posisi elemen UI secara fleksibel dengan menggunakan constraint (batas) terhadap parent atau elemen lainnya. Tiga atribut `xmlns` didefinisikan: `android` untuk atribut dasar Android, `app` untuk atribut khusus dari library AndroidX, dan `tools` untuk keperluan preview di Android Studio. Atribut `tools:context=".BookFragment"` menunjukkan bahwa layout ini digunakan oleh `BookFragment`, sehingga Android Studio bisa menampilkan preview sesuai konteks fragment tersebut.

Pada baris [10–17], didefinisikan sebuah `RecyclerView` dengan ID `@+id/recyclerView`. Komponen ini digunakan untuk menampilkan daftar item buku secara efisien dalam bentuk scrollable list. Lebar (`layout_width`) dan tinggi (`layout_height`) diset ke `0dp` agar ukuran akan

ditentukan oleh constraint. Komponen ini diposisikan menggunakan constraint: bagian atas dan bawah ditautkan ke Top dan Bottom parent, sedangkan bagian kiri (Start) dan kanan (End) juga ditautkan ke parent, sehingga RecyclerView akan memenuhi seluruh ruang dalam ConstraintLayout.

fragment_detail.xml:

Pada baris [1], terdapat versi XML dan encoding yang digunakan.

Pada baris [2–7], didefinisikan ScrollView sebagai root layout, yang memungkinkan konten di dalamnya dapat digulir secara vertikal. Layout ini menggunakan match_parent untuk lebar dan tinggi agar mengisi seluruh layar. ScrollView ini diberi latar belakang warna gelap #1C1C1C dan padding sebesar 16dp. Selain itu, terdapat deklarasi xmlns standar untuk namespace Android, serta namespace app untuk atribut khusus AndroidX seperti layout_constraint....

Pada baris [9–12], terdapat LinearLayout sebagai child langsung dari ScrollView, dengan orientasi vertikal. Artinya, semua elemen UI di dalamnya akan disusun dari atas ke bawah. Lebar layout diatur agar mengisi seluruh layar, dan tingginya menyesuaikan konten (wrap_content).

Pada baris [14–84], terdapat ConstraintLayout di dalam LinearLayout. Layout ini digunakan agar setiap elemen UI bisa diposisikan secara fleksibel berdasarkan constraint ke elemen lain. ConstraintLayout ini menggunakan lebar match_parent dan tinggi wrap_content, sehingga akan menyesuaikan tinggi total komponen yang ada di dalamnya.

Pada baris [18–25], terdapat CardView dengan ID card_image, yang digunakan sebagai wadah untuk menampilkan gambar buku dengan sudut melengkung. Atribut cardCornerRadius="8dp" membuat sudut dari CardView menjadi agak membulat. Posisi CardView ini dikunci (constraint) ke atas dan sisi kiri dan kanan parent. Di dalamnya terdapat ImageView.

Pada baris [27–32], terdapat ImageView dengan ID bookImage yang digunakan untuk menampilkan gambar buku. Ukuran gambar ditentukan sebesar 350dp x 520dp, dan menggunakan scaleType="centerCrop" agar gambar memenuhi ruang yang disediakan secara proporsional. Gambar default yang digunakan adalah @drawable/image1.

Pada baris [35–47], didefinisikan TextView dengan ID bookTitle yang menampilkan judul buku. TextView ini menggunakan ukuran teks 24sp dan style teks bold, dengan warna teks putih. Posisi komponen ini dikunci di bawah dari card_image, dimulai dari kiri parent, dan berakhir sebelum bookYear, sehingga judul akan berada di sebelah kiri tahun buku dengan proporsi otomatis.

Pada baris [49–59], terdapat TextView lain dengan ID bookYear, yang menampilkan tahun terbit buku. Ukuran teks sama seperti judul buku yaitu 24sp, dengan warna putih. Letaknya dikunci agar berada di kanan dari bookTitle, serta berada di bawah gambar. Penempatan ini membuat bookYear sejajar horizontal dengan bookTitle, tetapi di sisi kanan.

Pada baris [61–70], TextView dengan ID textTentang digunakan sebagai label kecil bertuliskan “Tentang”, sebagai penanda sebelum menampilkan deskripsi buku. Ukuran teks adalah 14sp dengan warna putih. Komponen ini diletakkan di bawah bookTitle dan berada di sisi kiri layar, mengikuti parent.

Pada baris [72–83], TextView dengan ID bookAbout digunakan untuk menampilkan deskripsi atau sinopsis buku. Ukuran teks kecil yaitu 14sp, dengan warna putih. Komponen ini dikunci posisinya tepat di bawah textTentang, dimulai dari kiri parent dan berakhir di sisi kanan parent. Karena tinggi layout wrap_content, maka isi deskripsi akan menyesuaikan panjang teks. Margin atas dan bawah digunakan untuk memberi jarak dari elemen lain.

Item_book.xml:

Pada baris [1], terdapat versi XML dan encoding yang digunakan.

Pada baris [2–125], layout utama menggunakan ConstraintLayout yang fleksibel dalam menempatkan elemen berdasarkan constraint. Lebar nya diset match_parent, artinya akan mengisi lebar parent-nya, dan tingginya wrap_content, jadi hanya sebesar konten di dalamnya.

Pada baris [8–24], terdapat CardView sebagai kontainer utama yang membungkus semua isi item buku. Ini memberikan efek melengkung pada sudutnya (cardCornerRadius="16dp") dan latar belakang abu gelap (@color/dark_gray). CardView ini diberi margin 8dp agar tidak mepet ke item lain di RecyclerView. Constraint-nya mengatur agar tetap di tengah-tengah parent layout.

Pada baris [19–123], di dalam CardView luar terdapat ConstraintLayout kedua yang mengatur elemen-elemen detail buku.

Pada baris [23–33], terdapat CardView kecil (ID: card_image) yang berisi gambar buku (ImageView). CardView ini memiliki cardCornerRadius="8dp" agar gambar memiliki sudut melengkung. Di dalamnya, ImageView berukuran 123dp x 200dp menampilkan gambar buku dengan scaleType="centerCrop" agar gambar memenuhi ruang secara proporsional.

Pada baris [43–55], TextView dengan ID bookTitle menampilkan judul buku. Ukurannya besar (24sp), bold, dan berwarna putih. Constraint-nya menempatkan teks ini di kanan gambar, dan tidak sampai ke tepi kanan karena akan ada elemen lain di sana.

Pada baris [57–66], TextView dengan ID bookYear menampilkan tahun terbit buku, berada di kanan bookTitle, dan dekat tepi kanan parent.

Pada baris [68–79], TextView dengan ID textTentang menampilkan label “Tentang”. Letaknya tepat di bawah bookTitle, dan berada di sebelah kiri bookAbout.

Pada baris [81–93], TextView dengan ID bookAbout berisi sinopsis atau deskripsi buku. Letaknya di sebelah kanan textTentang, dan posisinya di bawah bookTitle. Lebar ditentukan 184dp, agar cukup panjang untuk menampilkan kalimat pendek atau potongan sinopsis.

Pada baris [95–107], terdapat Button dengan ID btnBuy untuk aksi “Beli”. Tombol ini posisinya di bawah bookAbout dan di sebelah kiri tombol detail. Ukurannya fleksibel (wrap_content) dengan teks dari string resource beli_button.

Pada baris [109–122], tombol btnAbout untuk membuka detail buku. Letaknya di sebelah kanan tombol beli, dan juga di bawah bookAbout. Tombol ini akan memicu perpindahan ke halaman detail jika diklik.

Compose:

MainActivity.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–30], diimport berbagai komponen yang diperlukan untuk aktivitas dan tampilan aplikasi. ComponentActivity diimport untuk membuat activity berbasis komponen, yang digunakan untuk menginisialisasi dan mengatur UI dalam aplikasi Android menggunakan Jetpack Compose. setContent digunakan untuk menetapkan tampilan UI aplikasi, sementara

`enableEdgeToEdge` memungkinkan tampilan aplikasi untuk menggunakan seluruh area layar, mengabaikan area tepi yang biasanya terblokir oleh status bar atau navigation bar. Selain itu, berbagai fungsi dan class seperti `Surface`, `Text`, `Modifier`, `ViewModel`, `collectAsState`, `NavHost`, `composable`, dan lainnya diimport untuk membangun dan mengelola UI serta navigasi menggunakan Jetpack Compose. `BookDetailScreen` dan `BookListScreen` digunakan sebagai `composable` utama yang menampilkan daftar dan detail buku. `BookListTheme` digunakan sebagai tema utama aplikasi.

Pada baris [32], `MainActivity` dideklarasikan sebagai turunan dari `ComponentActivity`. Class ini merupakan titik awal aplikasi yang dijalankan ketika aplikasi dibuka. Sebagai aktivitas utama, `MainActivity` bertanggung jawab untuk mengatur siklus hidup aplikasi dan menampilkan antarmuka pengguna menggunakan Jetpack Compose.

Pada baris [33–41], dalam method `onCreate()`, kita memanggil `super.onCreate(savedInstanceState)` untuk memastikan siklus hidup activity dijalankan sesuai dengan standar. Kemudian, `enableEdgeToEdge()` dipanggil untuk memungkinkan konten tampilan memenuhi seluruh layar, tanpa ada pembatasan dari status bar. Selanjutnya, `setContent` digunakan untuk menetapkan tampilan UI aplikasi yang menggunakan tema `BookListTheme`, yang didefinisikan dalam file tema dan memastikan konsistensi style visual aplikasi. Di dalamnya, kita memanggil fungsi `BooksApp()` yang akan menampilkan tampilan utama aplikasi.

Pada baris [45–78], terdapat deklarasi fungsi `BooksApp` untuk mengelola struktur UI utama aplikasi dengan menggunakan Jetpack Compose. Fungsi ini diatur agar dapat bekerja dengan sistem navigasi dan memberikan tampilan daftar buku dan detail buku yang dapat dipilih oleh pengguna.

Pada baris [46], variabel `layoutDirection` dideklarasikan untuk mendapatkan informasi tentang arah tata letak UI. Variabel ini berguna untuk mengelola arah tata letak komponen, terutama ketika mendukung bahasa RTL (Right-to-Left), yang mempengaruhi cara elemen-elemen UI diposisikan.

Pada baris [47], variabel `navController` dideklarasikan dan diinisialisasi dengan `rememberNavController()`, yang menyediakan kontrol untuk navigasi antar layar dalam aplikasi. `navController` digunakan untuk mengelola transisi antar tampilan atau komponen dalam aplikasi, seperti berpindah dari tampilan daftar buku ke tampilan detail buku.

Pada baris [48], variabel `viewModel` dideklarasikan dan diinisialisasi menggunakan fungsi `viewModel()`, yang mengambil instance dari `BookListViewModel`. `ViewModel` ini digunakan untuk menyimpan dan mengelola data yang diperlukan oleh UI.

Pada baris [49], variabel `bookList` dideklarasikan untuk mengobservasi `StateFlow` dari `viewModel.booksList` menggunakan fungsi `collectAsState()`. Variabel ini akan secara otomatis memperbarui UI saat data buku berubah.

Pada baris [50], variabel `selectedBook` dideklarasikan untuk mengobservasi `StateFlow` dari `viewModel.selectedBook` menggunakan `collectAsState()`. Data ini akan digunakan untuk menampilkan detail buku yang dipilih oleh pengguna.

Pada baris [52–76], `Surface` dideklarasikan sebagai komponen pembungkus untuk menampilkan konten UI, dengan menggunakan modifier untuk mengatur ukuran dan padding. `fillMaxSize()` membuat `Surface` mengambil seluruh ruang yang tersedia di layar, sementara `statusBarsPadding()` memberi ruang di bagian atas layar agar konten tidak tertutup oleh status bar. Kemudian, `padding()` digunakan untuk menambahkan padding di sisi kiri dan kanan berdasarkan tata letak yang aman (*safe drawing area*), yang memastikan konten tidak terhalang oleh elemen UI seperti status bar atau navigation bar.

Pada baris [61–75], `NavHost` dideklarasikan untuk mengelola navigasi antar tampilan. `startDestination` ditetapkan ke `"list"`, yang berarti aplikasi akan memulai dengan menampilkan tampilan daftar buku. Di dalam `NavHost`, ada dua route yang didefinisikan. Route `"list"` akan memanggil `BookListScreen`, dengan daftar buku yang berasal dari variabel `bookList`. Ketika pengguna mengklik tombol detail, fungsi `onDetailClick` akan memanggil `viewModel.selectBook()` untuk memilih buku berdasarkan index, dan kemudian menavigasi ke tampilan `"detail"`. Route `"detail"` akan memanggil `BookDetailScreen` jika `selectedBook` tidak null. Jika data buku tidak ditemukan, maka akan ditampilkan pesan “Buku tidak ditemukan” menggunakan `Text`.

Datasource.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3-4], diimport resource `R` yang merujuk pada file resource aplikasi, seperti `strings.xml` dan `drawable`. Import ini memungkinkan kita untuk mengakses berbagai resource yang ada dalam aplikasi, seperti string untuk teks, gambar untuk `drawable`, dan lainnya.

Selain itu, diimport juga Book dari model `com.example.booklist.model.Book`. Book adalah class model yang menyimpan informasi tentang buku, seperti judul, tahun terbit, deskripsi, URL, dan gambar sampul yang akan digunakan dalam aplikasi untuk menampilkan data buku.

Pada baris [6], dideklarasikan class `Datasource`, yang bertugas untuk menyediakan data buku. Class ini merupakan bagian dari arsitektur aplikasi yang bertanggung jawab untuk mengelola dan menyediakan data yang diperlukan. `Datasource` biasanya digunakan untuk memisahkan logika pengambilan data dari logika tampilan atau UI, sehingga kode aplikasi lebih modular dan mudah dikelola. Dalam hal ini, class `Datasource` menyediakan data buku yang akan digunakan oleh fragment atau activity dalam aplikasi.

Pada baris [7–18], terdapat method `loadBooks()` yang berfungsi untuk mengembalikan sebuah list yang berisi objek `Book`. Dalam method ini, kita menggunakan `listOf<Book>()` untuk membuat sebuah list baru yang berisi objek-objek `Book`. Setiap objek `Book` dibentuk dengan memasukkan berbagai ID resource dari file `strings.xml` dan `drawable`. Misalnya, untuk judul buku, kita menggunakan `R.string.book_title1` yang merujuk pada string dengan nama `book_title1` di `strings.xml`. Begitu juga dengan tahun terbit, deskripsi, URL, dan gambar sampul buku yang diambil dari file `strings.xml` dan folder `drawable`. Dengan cara ini, aplikasi kita menjadi lebih fleksibel karena kita bisa mengelola data di file resource dan tidak perlu mengubah kode sumber setiap kali ada perubahan pada data buku. List yang berisi data buku ini kemudian dapat digunakan oleh komponen lain dalam aplikasi, seperti adapter di `RecyclerView`, untuk menampilkan daftar buku ke pengguna.

Book.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–4], diimport dua anotasi penting dari `AndroidX`: `@DrawableRes` dan `@StringRes`. Kedua anotasi ini digunakan untuk menunjukkan bahwa nilai parameter merupakan ID dari resource, bukan data literal.

Pada baris [6–12], didefinisikan sebuah data class bernama `Book`. Class ini digunakan untuk merepresentasikan data sebuah buku dalam bentuk struktur objek yang sederhana dan otomatis menyediakan fungsi-fungsi dasar seperti `toString()`, `equals()`, dan `hashCode()` karena merupakan data class. `titleResourceId`, `yearResourceId`, `aboutResourceId`, dan

`webViewResourceId` bertipe `Int` dan diberi anotasi `@StringRes`, menandakan bahwa nilainya merujuk pada string resource. Sedangkan `imageResourceId` bertipe `Int` dan diberi anotasi `@DrawableRes`, menandakan bahwa ini adalah ID dari resource gambar.

BookDetailScreen.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–32], diimport komponen penting dari Android dan Jetpack Compose. `Configuration` digunakan untuk mendeteksi orientasi layar. Komponen tata letak seperti `Row`, `Column`, `Spacer`, `Image`, serta atribut ukuran seperti `size`, `width`, `height`, dan `fillMaxSize` diambil dari `androidx.compose.foundation.layout`. `RoundedCornerShape` digunakan untuk memberi sudut membulat pada gambar. `MaterialTheme` dan `Text` digunakan untuk styling teks sesuai Material Design 3. `Modifier`, `Alignment`, `ContentScale`, `clip`, dan `verticalScroll` digunakan untuk mengatur tampilan dan perilaku UI. `painterResource` dan `LocalContext` digunakan untuk mengambil gambar dan teks dari resource aplikasi. Satuan ukuran seperti `dp` dan `sp` digunakan untuk mengatur dimensi dan ukuran huruf. Objek `Book` diimport dari package model sebagai data yang ditampilkan. `Timber` digunakan untuk mencatat log aktivitas saat detail buku ditampilkan.

Pada baris [35–80], dideklarasikan fungsi `BookDetailScreen()` sebagai composable yang menerima satu parameter `book` bertipe `Book`. Fungsi ini bertugas menampilkan halaman detail dari sebuah buku tertentu, termasuk gambar, judul, tahun, dan deskripsi buku.

Pada baris [36], log dicetak menggunakan `Timber.i` untuk menandai bahwa detail dari buku yang dimaksud sedang ditampilkan. Teks log berisi string judul buku yang diambil dari resources dengan menggunakan `LocalContext.current.getString()`.

Pada baris [37–42], digunakan layout `Column`, yang menyusun elemen UI secara vertikal dari atas ke bawah. `Modifier` yang digunakan adalah `fillMaxSize()` agar komponen memenuhi seluruh layar, `padding 16.dp` untuk memberi ruang tepi, dan `verticalScroll` agar konten dapat digulir jika melebihi tinggi layar.

Pada baris [43–51], ditampilkan gambar buku menggunakan komponen `Image`. Gambar dimuat dari resources menggunakan `painterResource(book.imageResourceId)`. Ukuran gambar diatur sebesar lebar `380.dp` dan tinggi `550.dp`. Gambar dibulatkan menggunakan `clip(RoundedCornerShape(16.dp))` dan disejajarkan secara horizontal di tengah dengan

`align(Alignment.CenterHorizontally)`. Properti `contentScale = ContentScale.Crop` digunakan agar gambar memenuhi ukuran yang ditentukan secara proporsional.

Pada baris [53], `Spacer` digunakan untuk memberikan jarak vertikal sebesar 16.dp antara gambar dan teks di bawahnya.

Pada baris [55–78], digunakan `Column` untuk menyusun teks informasi buku secara vertikal. `Modifier.fillMaxWidth()` digunakan agar kolom ini mengambil seluruh lebar yang tersedia. Atribut `horizontalAlignment = Alignment.Start` digunakan untuk menyelaraskan isi kolom ke sebelah kiri.

Pada baris [59–70], digunakan `layout Row` untuk menyusun judul buku dan tahun terbit secara horizontal di dalam satu baris. Atribut `horizontalArrangement = Arrangement.SpaceBetween` digunakan agar kedua komponen memiliki jarak antar ujung. Judul buku ditampilkan di sebelah kiri menggunakan `Text` dengan teks yang diambil dari `book.titleResourceId`, menggunakan style `MaterialTheme.typography.labelLarge` dan ukuran huruf 26.sp.

Pada baris [65–69], tahun terbit buku ditampilkan menggunakan `Text` di sisi kanan, dengan teks dari `book.yearResourceId`, menggunakan style `MaterialTheme.typography.bodyMedium` dan ukuran huruf 20.sp.

Pada baris [71], `Spacer` ditambahkan untuk memberi jarak vertikal sebesar 8.dp sebelum bagian deskripsi.

Pada baris [72], label teks “Tentang:” ditampilkan sebelum deskripsi buku, menggunakan style teks kecil `MaterialTheme.typography.bodySmall`.

Pada baris [73–77], deskripsi buku ditampilkan menggunakan `Text` dengan isi teks yang diambil dari `book.aboutResourceId`. Teks ini ditampilkan menggunakan style `MaterialTheme.typography.bodySmall` dan ukuran huruf 14.sp.

BookListScreen.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–38], diimport komponen penting dari Android dan Jetpack Compose. `Intent` dan `Uri` dari `android.content` dan `android.net` digunakan untuk membuka URL eksternal. Komponen tata letak seperti `Row`, `Column`, `Spacer`, `Image`, dan `LazyColumn` diambil dari `androidx.compose.foundation.layout` dan `foundation.lazy`. Untuk tampilan material modern,

digunakan Card, Button, Text, dan MaterialTheme dari `androidx.compose.material3`. Elemen tambahan seperti `PainterResource`, `StringResource`, `LocalContext`, serta atribut tampilan (`clip`, `ContentScale`, dan `Alignment`) mendukung pengambilan resource dan pengaturan gambar. `Preview`, `Modifier`, dan unit ukuran seperti `dp` dan `sp` juga digunakan untuk mendukung tampilan dan pengujian UI. Terakhir, data Book diimport dari package model dan class `BooksApp` disiapkan dari package utama aplikasi. `Timber` digunakan untuk mencatat log saat tombol ditekan.

Pada baris [41–50], didefinisikan fungsi composable `BookListScreen()` yang menerima tiga parameter: `bookList` berupa daftar buku (`List<Book>`), `onDetailClick` yaitu lambda untuk menangani klik tombol detail, dan modifier opsional untuk menyesuaikan tampilan dari luar. Pada baris [42], digunakan `LazyColumn` sebagai layout vertikal yang mendukung pemrosesan daftar besar secara efisien. Modifier disisipkan jika diperlukan dari luar composable ini.

Pada baris [43–48], digunakan fungsi `itemsIndexed()` untuk mengiterasi daftar indeks dari `bookList`. Untuk setiap index, pertama dicetak log menggunakan `Timber.d()` untuk menandai bahwa tombol “Detail” ditekan untuk buku tertentu. Kemudian dipanggil composable `BookCard`, di mana objek `bookList[index]` dikirim sebagai argumen. Parameter `onDetailClick` juga diteruskan, membungkus nilai index agar dapat dikenali saat tombol diklik.

Pada baris [53–135], dideklarasikan composable `BookCard()`, yang bertugas menampilkan kartu berisi data singkat dari sebuah buku. Fungsi ini menerima `book` sebagai parameter utama, `onDetailClick` untuk navigasi ke detail, dan modifier opsional.

Pada baris [54–58], digunakan Card dari Material 3 sebagai kontainer utama yang memiliki padding sebesar 8.dp dan bentuk sudut membulat (`RoundedCornerShape(16.dp)`). Modifier `fillMaxWidth()` memastikan kartu mengisi lebar layar.

Pada baris [60], variabel `context` diset sebagai `LocalContext.current` agar dapat digunakan untuk mengambil string resource atau memulai intent dari dalam fungsi composable.

Pada baris [65–132], layout `Row` digunakan untuk menyusun elemen secara horizontal. `Padding` ditetapkan 16.dp, dan `fillMaxWidth()` membuat baris mengisi lebar penuh.

Pada baris [66–74], ditampilkan gambar buku menggunakan `Image`. Resource gambar diakses dari ID dalam `book.imageResourceId` melalui `PainterResource()`. `contentDescription`

diberikan melalui `stringResource(book.aboutResourceId)` agar sesuai dengan prinsip aksesibilitas. Ukuran gambar diatur dengan `size(width = 120.dp, height = 200.dp)` dan bentuknya dibulatkan menggunakan `clip(RoundedCornerShape(12.dp))`. Gambar disejajarkan secara vertikal di tengah baris (`align(Alignment.CenterVertically)`) dan `ContentScale.Crop` menjaga rasio aspek gambar.

Pada baris [75], `Spacer` memberikan jarak horizontal 12.dp antara gambar dan kolom teks di sebelah kanan.

Pada baris [76–131], digunakan `Column` untuk menampilkan isi teks dan tombol dalam tata letak vertikal. Modifier `weight(1f)` membuat kolom ini memenuhi ruang horizontal yang tersisa, dan `align(Alignment.CenterVertically)` memastikan kolom berada di tengah baris secara vertikal.

Pada baris [81–98], `Row` di dalam kolom digunakan untuk menampilkan judul buku di sebelah kiri dan tahun terbit di kanan. `Arrangement.SpaceBetween` memastikan keduanya memiliki jarak yang merata di antara mereka. Judul buku ditampilkan dalam `Text` menggunakan resource string, dengan style `labelLarge`, ukuran huruf 26.sp, dan `overflow = TextOverflow.Ellipsis` agar teks panjang tidak meluber keluar. Tahun terbit ditampilkan di kanan dengan ukuran lebih kecil, 14.sp, dan style `bodyMedium`.

Pada baris [99], `Spacer` menambahkan jarak vertikal sebesar 20.dp sebelum deskripsi buku.

Pada baris [100–111], terdapat `Row` untuk memuat deskripsi buku ditampilkan dalam bentuk dua `Text`. Pertama teks label “Tentang:”, lalu diikuti oleh isi deskripsi dari resource string `book.aboutResourceId`. Keduanya menggunakan style `bodySmall` dan ukuran huruf 14.sp.

Pada baris [112], `Spacer` memberikan jarak sebesar 16.dp sebelum tombol aksi.

Pada baris [113–129], digunakan `Row` dengan `horizontalArrangement = Arrangement.End` agar tombol-tombol sejajar di sebelah kanan layar. Dua buah tombol ditampilkan: “Beli Buku” dan “Detail”.

Pada baris [115–122], tombol “Beli Buku” diberi aksi ketika diklik, yaitu mengambil URL dari `book.webUrlResourceId`, lalu membuat `Intent` dengan `ACTION_VIEW` dan membuka URL tersebut dengan `context.startActivity()` untuk membuka halaman pembelian buku melalui browser. Log aktivitas dicatat menggunakan `Timber.tag(...).d(...)`.

Pada baris [123], `Spacer` memberi jarak horizontal antara dua tombol sebesar 8.dp.

Pada baris [124–128], tombol “Detail” memanggil `onDetailClick()`, yaitu fungsi yang diteruskan dari luar composable dan biasanya mengarahkan pengguna ke halaman detail buku.

BookListViewModel.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–10], diimport beberapa komponen penting untuk mendukung arsitektur MVVM (Model-View-ViewModel) dan pemrosesan asynchronous menggunakan coroutine. `ViewModel` dan `viewModelScope` dari `androidx.lifecycle` digunakan untuk mengelola data secara lifecycle-aware. Class `Datasource` dan model `Book` diambil dari package aplikasi yang menangani data dan struktur objek buku. `MutableStateFlow` dan `StateFlow` dari `kotlinx.coroutines.flow` digunakan untuk mengelola dan mengamati state secara reaktif. Terakhir, `Timber` digunakan untuk mencatat log aktivitas.

Pada baris [12], dideklarasikan class `BookListViewModel` sebagai turunan dari `ViewModel`. Class ini memiliki satu parameter bernama `source` bertipe `String`, yang bisa digunakan untuk menentukan asal data atau untuk logging.

Pada baris [13–14], dibuat variabel privat `_booksList` bertipe `MutableStateFlow<List<Book>>`, yang menyimpan daftar buku dan diinisialisasi dengan list kosong. Versi publiknya, `booksList`, disediakan sebagai `StateFlow` yang hanya bisa dibaca, sehingga UI hanya dapat mengamati, tidak memodifikasi langsung nilainya.

Pada baris [16–17], dibuat variabel `_selectedBook` bertipe `MutableStateFlow<Book?>` untuk menyimpan satu buku yang dipilih. Nilainya bisa null. Sama seperti sebelumnya, variabel publik `selectedBook` disediakan dalam bentuk `StateFlow` untuk diamati UI tanpa bisa diubah secara langsung.

Pada baris [19–20], fungsi `init` dijalankan saat instance dari `ViewModel` ini pertama kali dibuat. Di dalamnya, dipanggil fungsi `loadBooks()` untuk memuat data buku.

Pada baris [20], dicatat log sumber data menggunakan `Timber.d()` dengan pesan yang menampilkan nilai parameter `source`. Ini berguna untuk debugging ketika ingin mengetahui dari mana data berasal.

Pada baris [24–30], didefinisikan fungsi privat `loadBooks()` yang digunakan untuk mengambil data dari sumber (`Datasource`). Pemanggilan dilakukan dalam coroutine

menggunakan `viewModelScope.launch`, agar proses berjalan asynchronous tanpa membekukan UI. Dalam coroutine, dipanggil `Datasource().loadBooks()` untuk mengambil daftar buku. Hasilnya disimpan ke dalam `_booksList.value`. Setelah berhasil, dicatat log jumlah item buku yang dimuat menggunakan `Timber.d()`.

Pada baris [32–35], didefinisikan fungsi publik `selectBook(book: Book)` untuk menyimpan buku yang dipilih pengguna ke dalam `_selectedBook`. Fungsi ini juga mencatat log informasi ID judul buku yang dipilih menggunakan `Timber.d()`, yang berguna untuk debugging atau pelacakan interaksi pengguna.

BookListViewModelFactory.kt:

Pada baris [1], dideklarasikan package tempat file ini berada, yaitu `com.example.booklist.screens`.

Pada baris [3–4], diimport class `ViewModel` dan `ViewModelProvider` dari library `Jetpack Lifecycle`. Keduanya digunakan untuk mengelola siklus hidup dan penyediaan instance `ViewModel` secara aman dan sesuai konteks lifecycle Android.

Pada baris [6], dideklarasikan class `BookListViewModelFactory` yang merupakan turunan dari interface `ViewModelProvider.Factory`. Class ini berfungsi untuk membuat instance `ViewModel` secara manual, khususnya saat `ViewModel` memerlukan argumen dalam konstruktor. Parameter `source` disiapkan dalam konstruktor meskipun belum digunakan.

Pada baris [7–12], override dilakukan pada fungsi `create()`. Fungsi ini memeriksa apakah class yang diminta (`modelClass`) adalah subclass dari `BookListViewModel`. Jika ya, maka dibuat dan dikembalikan instance `BookListViewModel(source)`, yaitu dengan meneruskan parameter `source`. Karena fungsi mengembalikan tipe generik `T`, maka diperlukan casting as `T` dan anotasi `@Suppress("UNCHECKED_CAST")` untuk menghindari peringatan dari compiler. Jika `modelClass` tidak sesuai, dilemparkan `IllegalArgumentException` dengan pesan class tidak dikenali.

MyApplication.kt:

Pada baris [1], ditentukan package tempat file ini berada, yaitu `com.example.booklist`.

Pada baris [3], diimport class Application dari Android SDK. Class ini merepresentasikan entry point global aplikasi dan digunakan untuk menginisialisasi komponen global saat aplikasi pertama kali dibuat.

Pada baris [4], diimport library Timber, yaitu library logging populer di Android yang mempermudah debugging dan pencatatan log.

Pada baris [7], dideklarasikan class MyApplication yang merupakan turunan dari class Application. Ini berarti class ini akan dijalankan sekali saat aplikasi pertama kali dimulai, sebelum aktivitas manapun dibuat.

Pada baris [8], method onCreate() dioverride. Method ini dipanggil oleh sistem saat aplikasi pertama kali dijalankan.

Pada baris [9], dipanggil super.onCreate() untuk memastikan inisialisasi dari class Application tetap dilakukan oleh Android framework.

Pada baris [11], dilakukan pengecekan terhadap BuildConfig.DEBUG. Jika aplikasi dalam mode debug (misalnya saat pengembangan), maka blok di dalam if akan dieksekusi.

Pada baris [12], dipanggil Timber.plant(Timber.DebugTree()), yang artinya mengaktifkan logger DebugTree milik Timber. Logger ini menampilkan informasi log secara lengkap (termasuk nama file, baris, dan method) yang sangat membantu untuk debugging selama pengembangan aplikasi.

Debugger

Debugger di Android Studio adalah tool yang digunakan untuk memeriksa dan memperbaiki kesalahan dalam aplikasi. Dengan menetapkan breakpoint (titik henti), kita bisa menghentikan sementara jalannya program tepat di baris yang ingin dianalisis. Saat aplikasi berhenti di titik itu, kita dapat melihat nilai variabel, urutan eksekusi, dan kondisi aplikasi saat itu. Ini sangat membantu untuk menemukan bug yang tidak terlihat hanya dari membaca kode atau log biasa. Setelah aplikasi berhenti, kita bisa menggunakan fitur Step Into untuk masuk ke dalam method yang dipanggil, Step Over untuk menjalankan satu baris kode tanpa masuk ke method lain, dan Step Out untuk keluar dari method yang sedang dianalisis dan kembali ke bagian kode sebelumnya. Dengan ketiga fitur ini, kita bisa melacak alur eksekusi aplikasi secara detail dan memahami bagaimana data mengalir di dalam program.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

[Praktikum_Mobile/Modul 4 at main · Bukhrii/Praktikum_Mobile](#)

SOAL 2

Soal Praktikum:

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

A. Pembahasan

Application class di Android berfungsi sebagai titik awal dari seluruh aplikasi dan hanya dijalankan satu kali selama aplikasi aktif. Class ini biasanya digunakan untuk mengatur konfigurasi global seperti inisialisasi library (misalnya Timber untuk logging atau Retrofit untuk jaringan), pengaturan database, atau menyimpan data yang dibutuhkan di banyak bagian aplikasi. Untuk menggunakannya, buat class baru yang mewarisi Application, lalu buat class tersebut di file AndroidManifest.xml melalui atribut android:name. Dengan cara ini, semua komponen penting bisa disiapkan di satu tempat dan tidak perlu diulang di setiap Activity atau Fragment agar membantu menjaga struktur aplikasi tetap rapi dan efisien.